



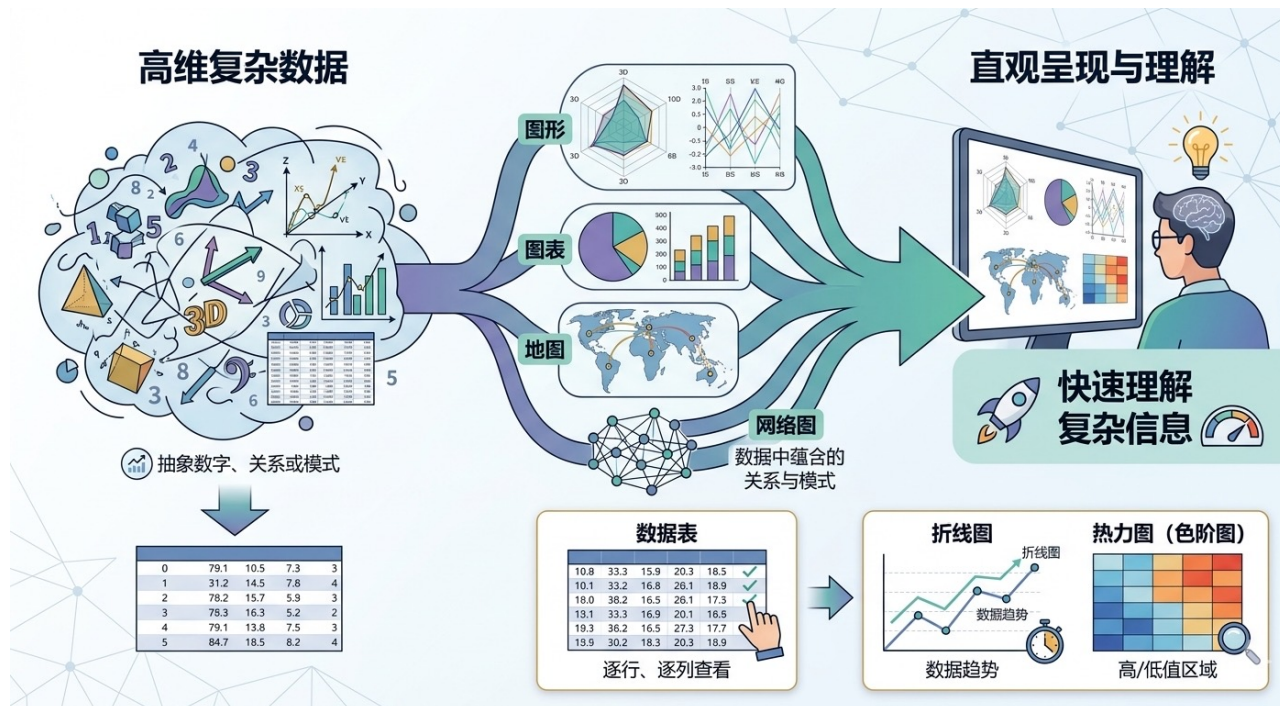
《实用Python编程》

第11讲: Matplotlib表格数据可视化

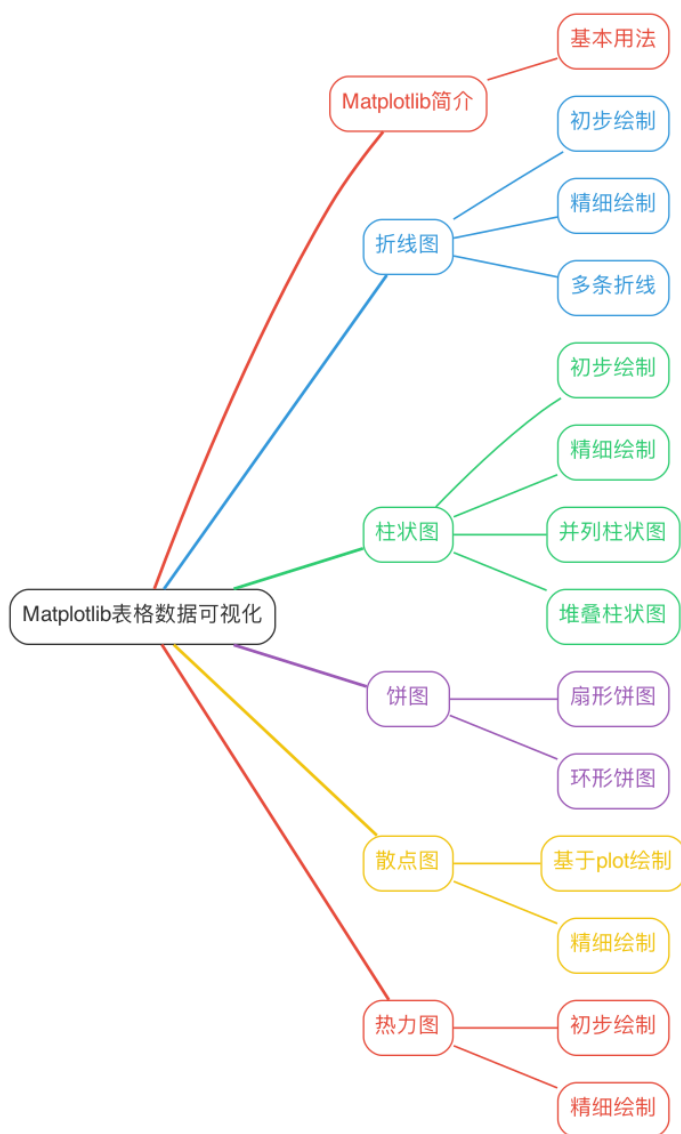
实用Python编程 教研组

数据可视化 (data visualization)

- 旨在将数据中蕴含的相对抽象的数字、关系或模式转化为直观的视觉形式，以帮助用户快速理解高维复杂信息

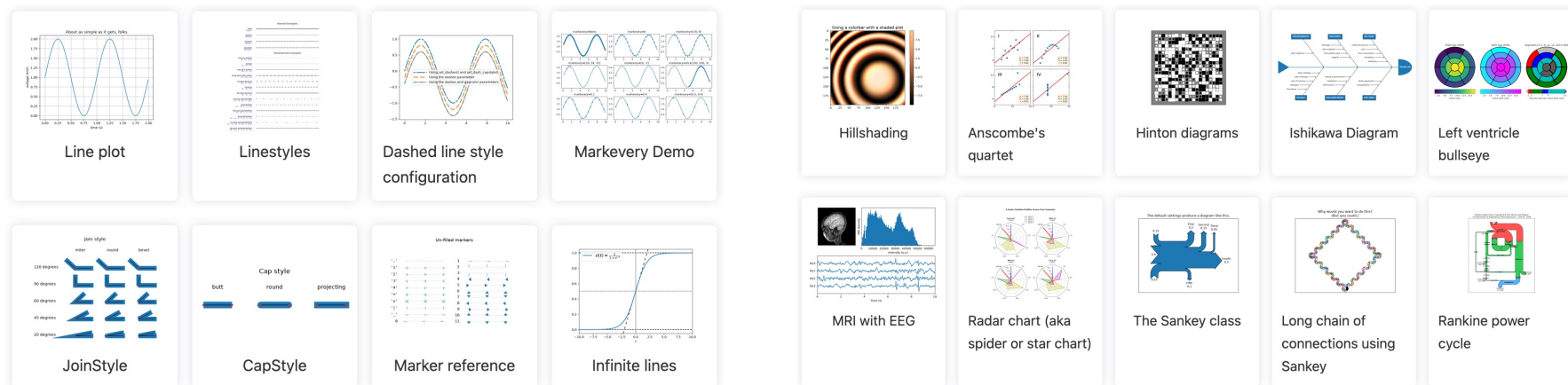


提纲



Matplotlib

- 最常用的表格数据可视化库
 - 少量代码即可实现出版物级别的图表绘制
 - 官网 <https://matplotlib.org/>
 - 安装指令 `pip install matplotlib`





基本用法

- 以绘制正弦曲线 $y=\sin(x)$ 为例

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'Heiti TC', 'Hiragino Sans GB', 'PingFang SC'] # 正常显示中文
4 plt.rcParams['axes.unicode_minus'] = False # 正常显示负号
5 # 添加魔法命令, 让图表直接显示在Notebook代码单元格下方
6 %matplotlib inline
```

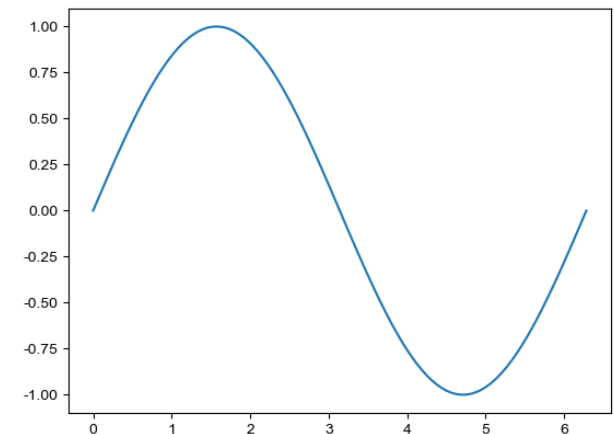
代码解读

- 第1-2行: Matplotlib一般与NumPy一起使用, 因此先导入这两个库
- 第3-4行: 修改配置rcParams (runtime configuration parameters) 以支持中文显示
- 第6行: 魔法命令已非必需, 添加该命令是一种稳妥做法

基本用法

- 以绘制正弦曲线 $y=\sin(x)$ 为例

```
1 fig, axes = plt.subplots() # 创建画布和子图
2
3 x = np.linspace(0, 2 * np.pi, 100)
4 y = np.sin(x)
5
6 axes.plot(x, y) # 调用子图的绘图功能
7
8 plt.savefig('sine.png') # 将绘图结果保存为本地图片文件
```



`fig`和`axes`是Matplotlib绘图中的两个基本对象

- `fig`对应一张画布 (canvas) , 默认尺寸是宽6.4英寸, 高4.8英寸
 - `plt.subplots(figsize=(12,10))`将创建一个宽12英寸, 高10英寸的画布
- `axes`是画布中的一个子图 (subfigure)
- 一次绘图过程中, 画布是唯一的, 而子图可以有多个
- 所有的绘图操作都在子图上进行, 因此`axes`也是“画笔”

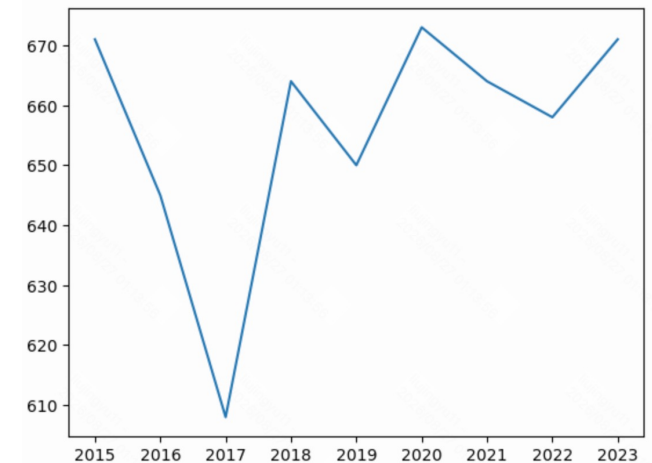
折线图 (line chart)

- 常用于观察数据变化趋势和规律
- 绘图函数: plot

```
1 fig, axes = plt.subplots()
2 years = list(range(2015, 2024))
3 scores = [671, 645, 608, 664, 650, 673, 664, 658, 671]
4 axes.plot(years, scores)
```

当前折线图的问题

- 坐标轴没有标签, 横、纵坐标含义不明
- 样本点数值不易观察
- 坐标刻度字体偏小



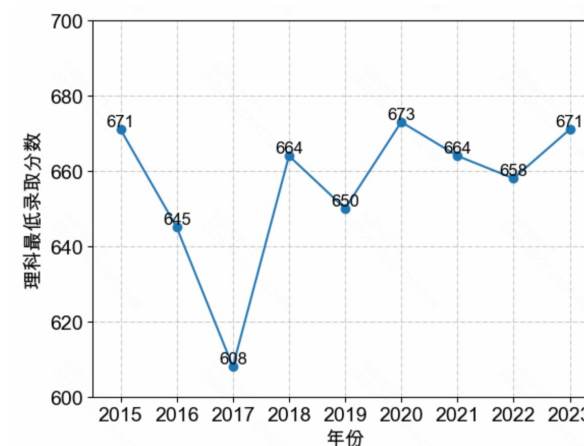
数据源: 2015-2023年中国人民大学工商管理专业在福建省理科的录取分数线



折线图：精细绘制

- 添加坐标轴标签、样本点数值、增大刻度字体、设置网格线

```
4 axes.plot(years, scores, '-o')
5 # 为每个样本点添加分数标识
6 for year, score in zip(years, scores):
7     axes.annotate(score, xy=(year, score), ha='center', va='bottom', fontsize=12)
8 # 设置坐标轴标签
9 axes.set_xlabel("年份", fontsize=14) #, labelpad=8) # 设置x轴标签
10 axes.set_ylabel("理科最低录取分数", fontsize=14) #, labelpad=8) # 设置y轴标签
11 # 设置坐标轴显示范围
12 axes.set_xlim(2014.5, 2023.5)
13 axes.set_ylim(600, 700)
14 # 设置x坐标轴刻度
15 axes.set_xticks(years)
16 # 设置横纵坐标轴刻度字体大小
17 axes.tick_params(axis='x', labelsz=14)
18 axes.tick_params(axis='y', labelsz=14)
19 # 设置网格线
20 axes.grid(linestyle='-.', linewidth=0.5)
```





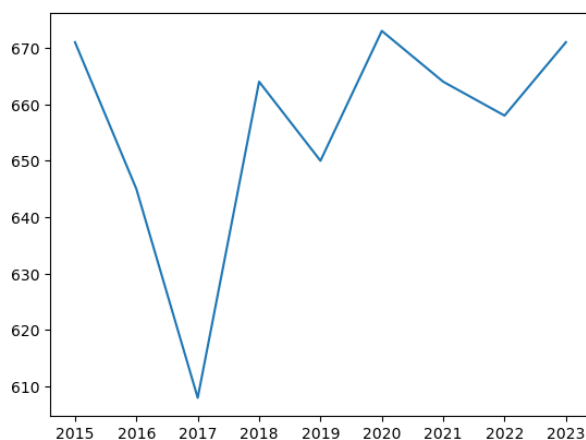
折线图：精细绘制

• 指定线型

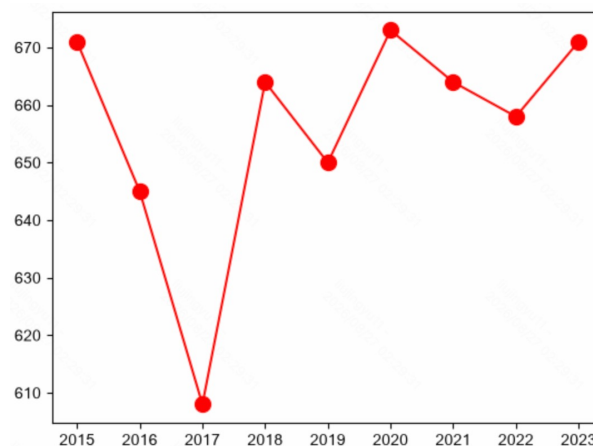
表 6.2: 折线绘制方法plot的参数

参数	默认值	作用
x	-	数据序列的横坐标
y	-	数据序列的纵坐标
color	-	折线的颜色，不设置时系统会自动指定
marker	None	数据点样式: None (不设置)、'o' (圆圈)、's' (方形)、'^' (三角形)、'+' (加号)、'*' (星号)、'd' (菱形) 等
markersize	6.0	数据点的大小
linestyle	'-'	线型, 包括 '-' (直线)、'--' (虚线)、'-.' (点划线)、':' (点线)
linewidth	1.5	线宽
label	-	图例中该折线的标签, 不设置则不会出现在图例中

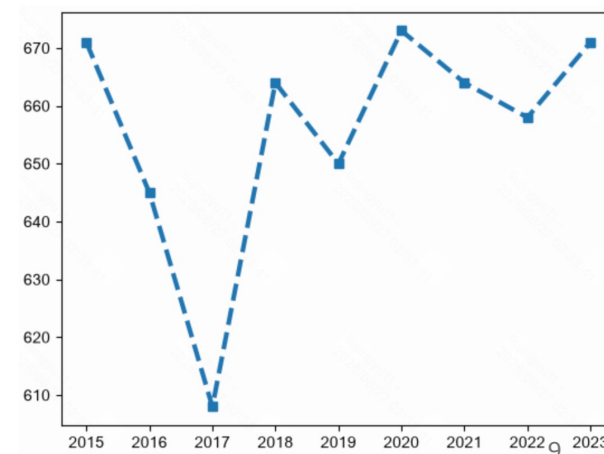
```
axes.plot(years, scores)
```



```
axes.plot(years, scores,  
color='red', marker='o',  
markersize=10)
```



```
axes.plot(years, scores,  
marker='s',linestyle='--',  
linewidth=3)
```



折线图：精细绘制

- 设置坐标样式

`axes.set_ylabel("理科最低录取分数",
fontsize=14)`

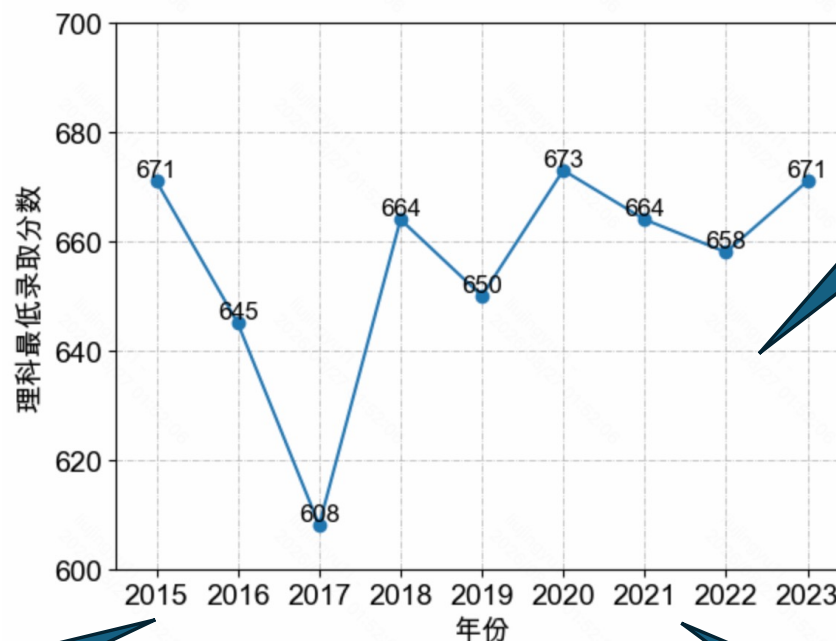
`axes.set_xlim(2014.5, 2023.5)`

`axes.set_xlabel("年份",
fontsize=14)`

`axes.grid(linestyle='-.',
linewidth=0.5)`

`axes.tick_params(axis='x',
labelsize=14)`

`axes.set_xticks(years)`



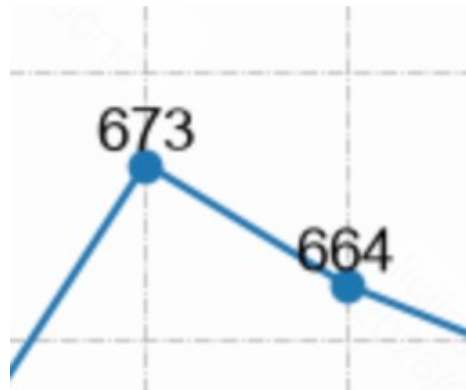
折线图：精细绘制

- 添加文本标注 (表6.3)

- xy: 文本坐标二元组(x,y)
- ha: “left”表示文本的左侧坐标为x
- va: “bottom”表示文本的底部侧坐标为y
- color: 文本颜色

```
axes.annotate(score, xy=(year,  
score), ha='center',  
va='bottom', fontsize=12)
```

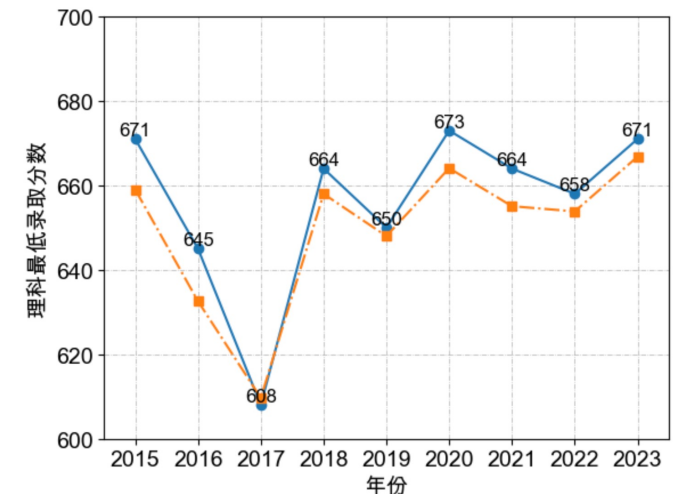
```
axes.annotate(score, xy=(year+0.1,  
score+0.1), ha='left',  
va='bottom', color='red')
```



绘制多条折线图

- 只要多次调用plot方法就可在同一张画布中画出多条折线
 - 第4行：绘制工商管理专业的录取分数线
 - 第6行：绘制平均分

```
3 scores = [671, 645, 608, 664, 650, 673, 664, 658, 671]
4 axes.plot(years, scores, '-o')
5 mean = [659, 632.56, 609.67, 658., 648, 664.09, 655.08, 653.85, 666.81]
6 axes.plot(years, mean, '-.s')
```



当前折线图的问题

- 如果只看图，两条折线的含义不明确

绘制多条折线图

- 添加图例 (legend) , 标识每条折线的含义

```

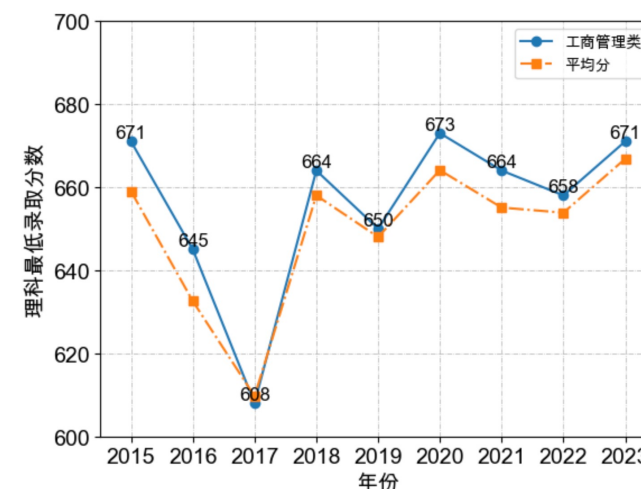
3 scores = [671, 645, 608, 664, 650, 673, 664, 658, 671]
4 axes.plot(years, scores, '-o', label='工商管理类')
5 mean = [659, 632.56, 609.67, 658., 648, 664.09, 655.08, 653.85, 666.81]
6 axes.plot(years, mean, '-.s', label='平均分')
7
8 axes.legend(loc='best', fontsize=10)

```

使用图例需设置label参数

表 6.4: 添加图例方法 legend 的参数设置

参数	默认值	功能讲解
loc	"best"	图例位置, 可设为"best" (自适应)、"upper left" (左上角)、"bottom right" (右下角) 等
handlelength	2.0	图例中的线长度
fontsize	12	图例中的字体大小
shadow	False	是否显示阴影效果

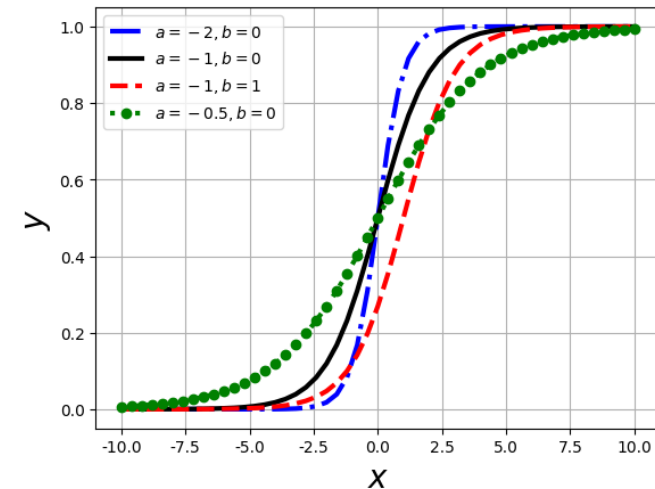


课堂练习-6-1-1

当参数a取负值时，下列函数可以将任意范围的数压缩为0到1之间的值：

$$y(x) = \frac{1}{1 + e^{(ax+b)}}$$

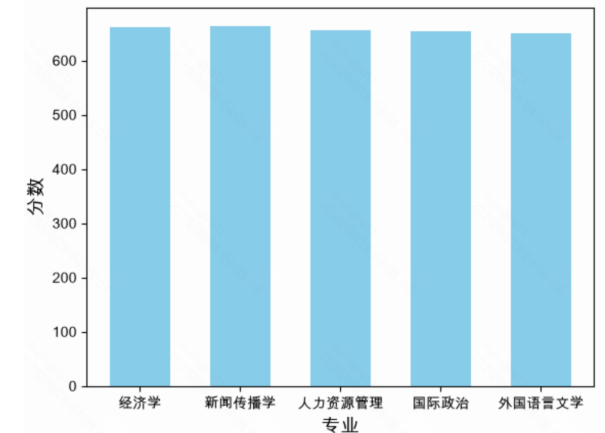
- 画出不同参数组合时的函数曲线，观察参数对于曲线形状、位置的影响
- 输入变量x的取值范围 $[-10, 10]$



柱状图 (bar chart)

- 常用于比较不同类别之间的数值大小
 - 比如展示特定年份不同专业的报考热度
- 绘图函数: bar

```
1 fig, axes = plt.subplots()
2 majors = ['经济学', '新闻传播学', '人力资源管理', '国际政治', '外国语言文学']
3 scores = np.array([661, 664, 656, 654, 651])
4 axes.bar(majors, scores, color='skyblue', width=0.6)
5 axes.set_xlabel("专业", fontsize=14)
6 axes.set_ylabel("分数", fontsize=14)
```



当前绘图的问题

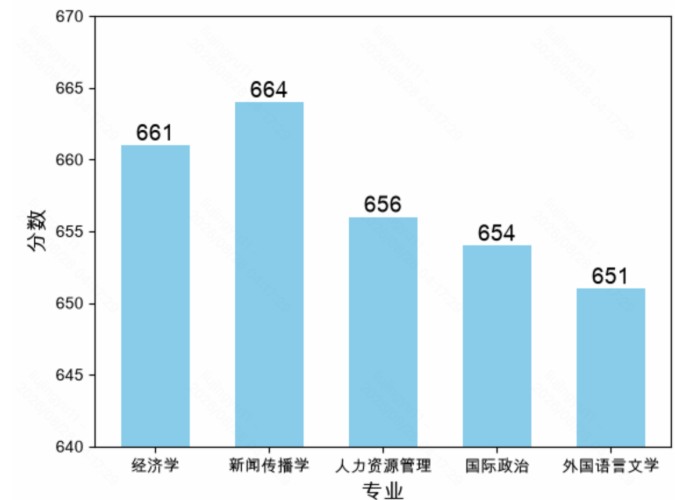
- 自动确定的坐标范围不适合观察数据

数据源: 2019年中国人民大学在北京市特定专业的录取分数

柱状图：精细绘制

- 调整坐标轴范围、添加数值标注
 - 通过`axes.set_ylim`手动调整y轴范围至[640,670]
 - 通过`axes.bar_label`在每个柱子上方添加对应的数值

```
1 fig, axes = plt.subplots()
2
3 bar = axes.bar(majors, scores, color='skyblue', width=0.6)
4
5 axes.bar_label(bar, fontsize=14) # 在每根柱子上方添加对应的数值
6
7 axes.set_xlabel("专业", fontsize=14)
8 axes.set_ylabel("分数", fontsize=14)
9
10 axes.set_ylim(640, 670) # 设置y轴范围
```

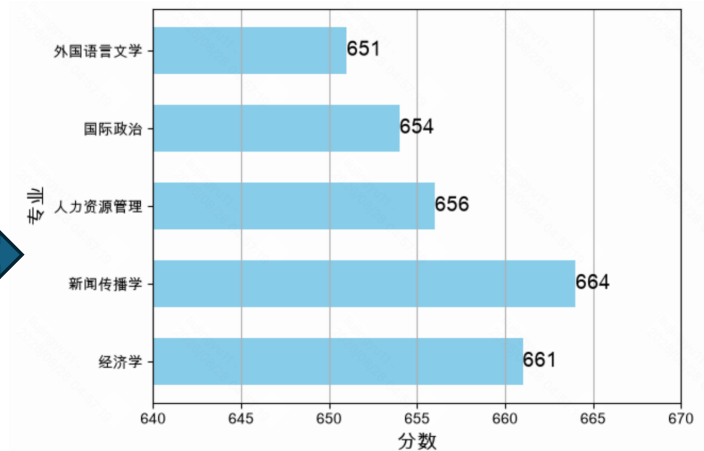


柱状图：精细绘制

- 用`barh`函数绘制水平方向的柱状图
 - 当类别（专业）较多时，水平柱状图更易空间布局
 - 通过`axes.grid(True, axis='x')`添加纵向网格线

```
1 fig, axes = plt.subplots()
2
3 bar = axes.barh(majors, scores, color='skyblue', height=0.6)
4 axes.bar_label(bar, fontsize=14) # 添加数值
5 axes.set_xlabel("分数", fontsize=14)
6 axes.set_ylabel("专业", fontsize=14)
7 axes.set_xlim(640, 670)
8 axes.grid(True, axis='x') # 添加网格线使数据更易读
```

试试将第8行注释掉或者去掉`axis='x'`



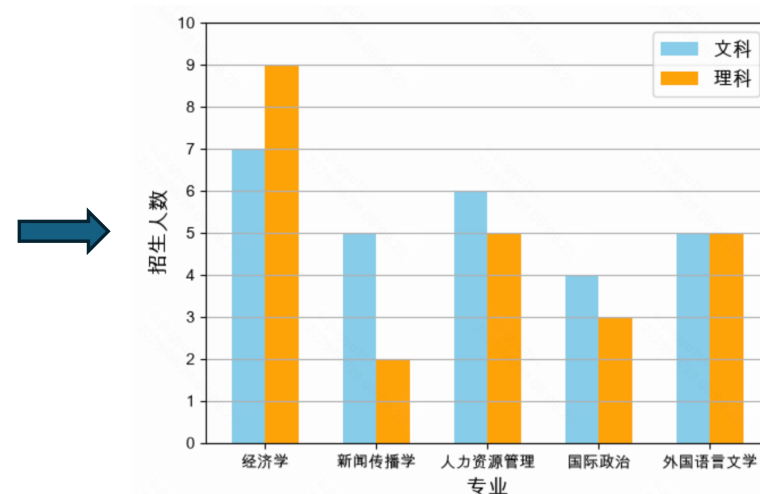
并列柱状图

- 用于比较多组数据
- 绘图要点
 - 用整数序列作为bar函数的x参数
 - 每组对应的序列值适当加减（第8-9行），使得不同组的柱状图错开

```

1 majors = ['经济学', '新闻传播学', '人力资源管理', '国际政治', '外国语言文学']
2 wenke = [7, 5, 6, 4, 5] # 文科招生人数
3 like = [9, 2, 5, 3, 5] # 理科招生人数
4
5 fig, axes = plt.subplots()
6 x = np.arange(len(majors))
7
8 axes.bar(x - 0.15, wenke, color='skyblue', width=0.3, label='文科') # 平移柱的水平位置并绘制
9 axes.bar(x + 0.15, like, color='orange', width=0.3, label='理科') # 平移柱的水平位置并绘制
10
11 axes.set_xlabel("专业", fontsize=14)
12 axes.set_ylabel("招生人数", fontsize=14)
13 axes.set_ylim(0, 10)
14 axes.set_yticks(ticks=range(11)) # 以1为步长, 显示细粒度刻度
15 axes.set_xticks(ticks=range(len(majors)))
16 axes.set_xticklabels(majors) # 将水平刻度改为专业名
17 axes.grid(True, axis='y') # 显示水平网格线
18 axes.legend(fontsize=12)
    
```

试试将第15-16行注释掉



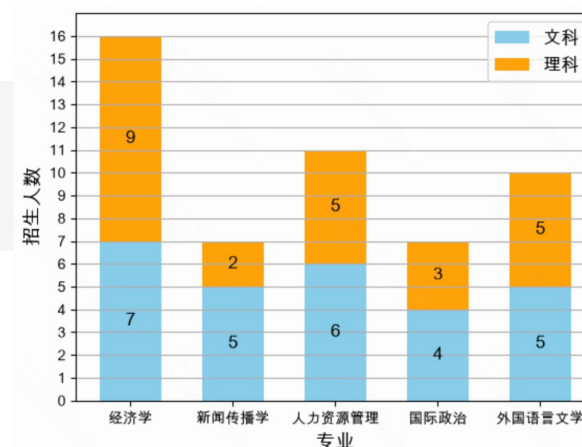
数据源：2019年人大在北京市各专业文理科招生人数



堆叠柱状图

- 在展示各专业文理科人数比例的同时，展示该专业的招生总数
- 绘图要点
 - 通过bar函数的bottom参数指定以文科组人数作为理科组柱状图的起点
 - 通过bar_label函数将人数居中显示在对应柱子上

```
8 wenke_bar = axes.bar(x, wenke, color='skyblue', width=0.6, label='文科')
9 like_bar = axes.bar(x, like, color='orange', width=0.6, bottom=wenke, label='理科')
10 axes.bar_label(wenke_bar, label_type='center', fontsize=12)
11 axes.bar_label(like_bar, label_type='center', fontsize=12)
```





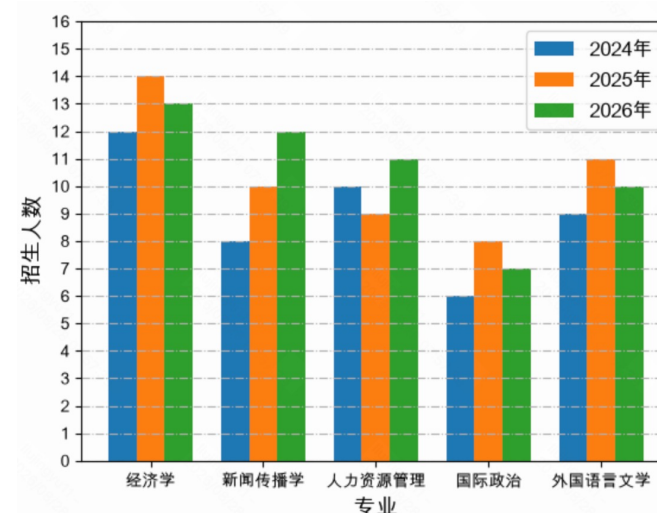
课堂练习-6-1-2

某大学若干专业在2024至2026之间的招生人数如下：

专业	2024	2025	2026
经济学	12	14	13
新闻传播学	8	10	12
人力资源管理	10	9	11
国际政治	6	8	7
外国语言文学	9	11	10

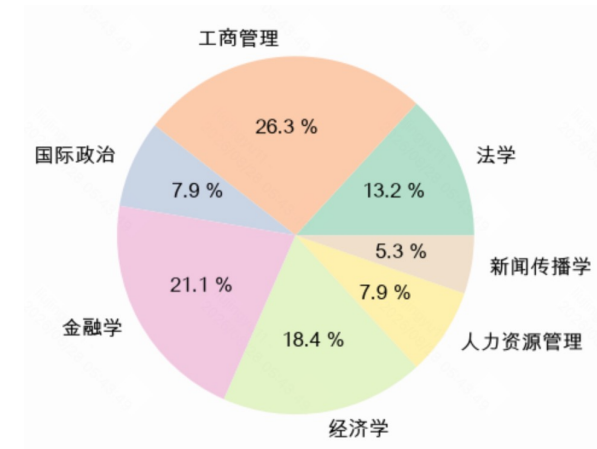
请以柱状图形式展示各专业各年的招生人数

样例仅供参考，可自由发挥



饼图 (pie chart)

- 常用于展示整体中各个成分的占比
 - 比如展示某年各专业新生的占比
- 绘图函数: pie



```
1 fig, axes = plt.subplots()
2 majors = ['法学', '工商管理', '国际政治', '金融学', '经济学', '人力资源管理', '新闻传播学']
3 numbers = [5, 10, 3, 8, 7, 3, 2]
4 colors = plt.color_sequences["Pastel2"]
5 axes.pie(x=numbers, labels=majors, colors=colors, autopct='%.1f%%', textprops={'fontsize': 12})
6 plt.show()
```

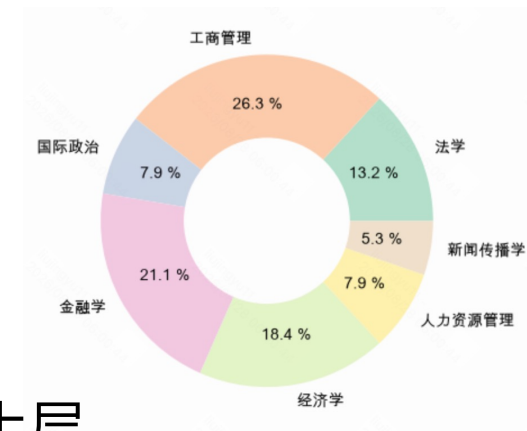
绘图要点

- x: 数值序列 (pie函数会根据这些数值自动计算出各成分的百分比并划分扇区)
- labels: 扇区标签序列, 需与x对齐
- colors: 扇区颜色序列 (教材图6.7)
- autopct: 百分比格式, 默认不显示 (%.1f表示保留小数点后1位显示, %%转义以显示百分号)
- textprops: 以字典形式设置扇区标签的字体及颜色

环形饼图

- 绘图要点

- 类似油画的原理，最晚绘制的图形会呈现在最上层
- 在绘制了之前的扇形饼图后，再绘制一个较小的、白色同心圆（第6行）
- 通过pctdistance参数适当调整扇区标签到圆心的距离（第5行）
 - 取值范围0~1，越小越接近圆心

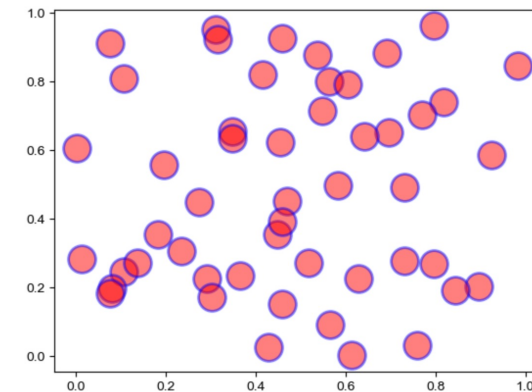


```
1 fig, axes = plt.subplots()
2 majors = ['法学', '工商管理', '国际政治', '金融学', '经济学', '人力资源管理', '新闻传播学']
3 numbers = [5, 10, 3, 8, 7, 3, 2]
4 colors = plt.color_sequences["Pastel2"]
5 axes.pie(x=numbers, labels=majors, colors=colors, autopct='%.1f%%', textprops={'fontsize': 12}, pctdistance=0.7)
6 axes.pie([1], colors=['white'], radius=0.5)
7 plt.show()
```

散点图 (scatter plot)

- 将多个二维数据点同时呈现在一个平面上，用于观察两个维度之间是否存在某种相关性
 - 比如数据点是否表现出聚集性，是否存在离群点等
- 绘图函数：沿用折线图函数plot
- 绘图要点：通过隐去样本点之间的连线，让样本呈散点状分布

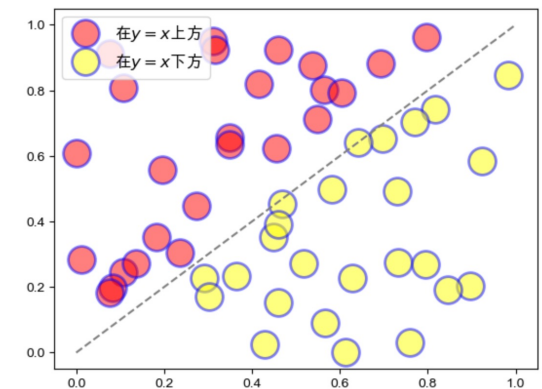
```
1 points = np.random.rand(50, 2) # 随机生成50个二维样本点
2 fig, axes = plt.subplots()
3 axes.plot(points[:,0], points[:,1],
4           linestyle='none', # 隐藏数据点之间的连线
5           marker='o', # 标记形状 (圆形)
6           markersize=20, # 标记大小
7           markerfacecolor='red', # 标记填充色
8           markeredgecolor='blue', # 边缘颜色
9           markeredgewidth=2, # 边缘宽度
10          alpha=0.5 # 填充色透明度
11         )
```



散点图：精细绘制

- 以对角线 $y=x$ 为分界线，用不同填充色指示落在分界线上方及下方的数据点
 - 用ndarray的布尔索引功能对数据点进行分组（第1-2行）
 - 本质上是二分类（第5章）
 - 为每组数据分别绘制散点图（第6-10行）

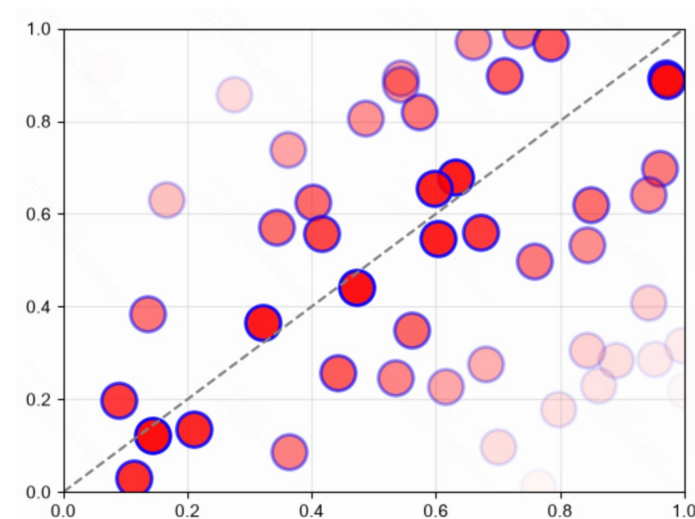
```
1 above = points[points[:,0]<points[:,1]] # 满足  $x < y$  的点落在直线  $y=x$  上方
2 below = points[points[:,0]>points[:,1]] # 满足  $x > y$  的点落在直线  $y=x$  下方
3
4 fig, axes = plt.subplots()
5
6 axes.plot(above[:,0], above[:,1], linestyle='none', marker='o', markersize=20, markerfacecolor='red',
7           markeredgecolor='blue', markeredgewidth=2, alpha=0.5, label="在 $y=x$ 上方") # 上方的点用红色填充
8
9 axes.plot(below[:,0], below[:,1], linestyle='none', marker='o', markersize=20, markerfacecolor='yellow',
10           markeredgecolor='blue', markeredgewidth=2, alpha=0.5, label="在 $y=x$ 下方") # 下方的点用黄色填充
11
12 x = np.linspace(0,1,50)
13 y = x
14 axes.plot(x, y, '--', color="gray") # 绘制直线  $y=x$ 
15
16 plt.legend(fontsize=12)
```



课堂练习-6-1-3

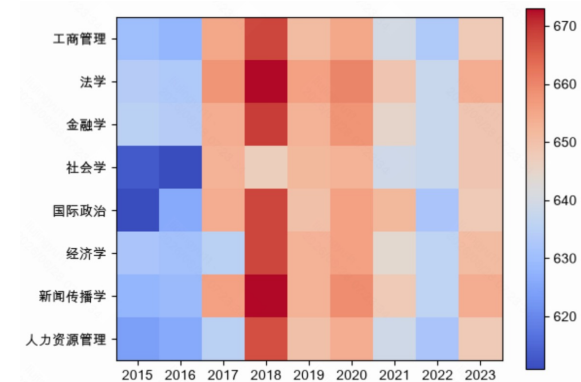
给定由`np.random.rand(50,2)`随机生成的50个数据点，绘制散点图

- 点的标记样式仍沿用示例代码，但要求距离直线 $y=x$ 越远的点，透明度越高
- 对于数据点 (u,v) ，其到 $y=x$ 的距离记为 d ，则有 $d = \frac{|u-v|}{\sqrt{2}}$
- 该点对应的透明度参数 $\alpha = (1 - \frac{d}{d_{\max}})^2$ ，其中 $d_{\max}$ 为距离最大值
- 可尝试给出不同的透明度参数计算公式，需满足
 - 与 d 成反比
 - 返回值区间 $[0,1]$



热力图 (heatmap)

- 用颜色显示表格数据的可视化技术
- 绘图函数: `imshow`



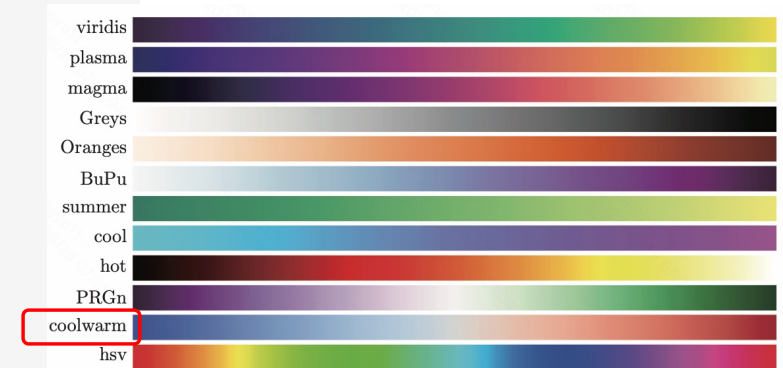
展示数据沿特定方向的变化模式及突变

```

1 majors = ['工商管理', '法学', '金融学', '社会学', '国际政治', '经济学', '新闻传播学', '人力资源管理']
2 years = np.arange(2015, 2024)
3 scores = np.array([[630, 628, 655, 668, 651, 655, 640, 633, 648],
4                    [634, 633, 658, 673, 656, 660, 649, 638, 654],
5                    [635, 634, 654, 669, 653, 658, 645, 638, 649],
6                    [613, 611, 653, 647, 652, 653, 639, 638, 649],
7                    [611, 626, 654, 668, 650, 656, 652, 632, 648],
8                    [632, 631, 635, 668, 653, 656, 644, 636, 651],
9                    [628, 629, 656, 673, 653, 659, 648, 636, 654],
10                   [624, 626, 635, 667, 650, 654, 639, 632, 648]])
11
12 fig, axes = plt.subplots()
13 heatmap = axes.imshow(scores, cmap='coolwarm')
14 cb = fig.colorbar(heatmap, ax=axes, aspect=20)
15 axes.set_xticks(np.arange(years.shape[0]))
16 axes.set_xticklabels(years)
17 axes.set_yticks(np.arange(len(majors)))
18 axes.set_yticklabels(majors)

```

colorbar反映颜色与数值之间的对应关系



Matplotlib 颜色映射表

数据源: 2015-2023年中国人民大学部分专业在湖南省的文科录取分数线

热力图：精细绘制

• 绘图要点

- 用`annotate()`将分数直接标到热度图的对应格子中（第10行）
- 文字颜色要根据格子背景色调整（第9行）
 - 根据每个格子对应的数值是否大于均值（`avg_val`），自动选择深色或浅色字体

```
1 avg_val = scores.mean()
2
3 fig, axes = plt.subplots()
4 heatmap = axes.imshow(scores, cmap='hot')
5
6 for i in range(years.shape[0]):
7     for j in range(len(majors)):
8         val = scores[j][i]
9         font_color = 'black' if val > avg_val else 'white'
10        axes.annotate(val, xy=(i, j), ha='center', va='center', color=font_color)
11
12 axes.set_xticks(np.arange(years.shape[0]))
13 axes.set_xticklabels(years)
14 axes.set_yticks(np.arange(len(majors)))
15 axes.set_yticklabels(majors)
```

